STM2 — Fault Detection Layer

By: Dhanush Raja A K

This layer runs on STM2 and is responsible for checking the sensor data received over CAN and deciding what action to take, controlling the fan, buzzer, and LCD based on which faults are active.

Fault Detection Flow

```
Receive CAN Data
      |
Check Under Voltage
      |
Check Over Voltage
      |
Check Over Current
      |
Check Over Temperature
      |
Check Smoke
      |
Check CAN Timeout
      |
Check CAN Bus-Off
      |
Check Sensor Failure
      |
Execute Action (Fan / Buzzer / LCD)
```

---

1. Under Voltage

```
if (voltage < 12.5)
    under_voltage_fault = 1;
else
    under_voltage_fault = 0;
```

Triggers when the battery is almost fully discharged.

Device	Action
Fan	OFF
Buzzer	Beep every 2 sec
LCD	UNDER VOLTAGE

---

2. Over Voltage

```
if (voltage > 16.8)
    over_voltage_fault = 1;
else
    over_voltage_fault = 0;
```

Triggers when the battery is being overcharged.

Device	Action
Fan	OFF

Buzzer	Continuous
LCD	OVER VOLTAGE

---

3. Over Current

```
if (current > 8.0)
    over_current_fault = 1;
else
    over_current_fault = 0;
```

Triggers when the load is drawing excessive current.

Device	Action
Fan	OFF
Buzzer	Slow beep
LCD	OVER CURRENT

---

4. Over Temperature

Uses hysteresis (separate ON and OFF thresholds) so the fan doesn’t rapidly switch on and off near the trigger point.

```
if (temperature >= 45) {
    fan = ON;
    over_temp_fault = 1;
}

if (temperature <= 40) {
    fan = OFF;
    over_temp_fault = 0;
}
```

Device	Action
Fan	ON
Buzzer	OFF
LCD	COOLING...

Critical temperature is checked separately, above 55°C the situation escalates beyond normal cooling:

```
if (temperature >= 55)
    critical_temp_fault = 1;
```

Device	Action
Fan	ON
Buzzer	Slow beep
LCD	BATTERY HOT

---

5. Smoke Detected

```
if (smoke_adc > 250)
    smoke_fault = 1;
else
```

```
    smoke_fault = 0;
```

Use whatever threshold matches your MQ135 calibration. This is the highest-priority fault in the system.

Device	Action
Fan	ON
Buzzer	Continuous
LCD	FIRE RISK

---

6. CAN Communication Timeout

STM1 sends data every 100 ms, so STM2 keeps a timer and flags a timeout if too much time passes without a new message:

```
if (Current_Time - Last_CAN_Receive_Time > 500)
    can_timeout_fault = 1;
```

Device	Action
Fan	OFF
Buzzer	Continuous
LCD	CAN TIMEOUT

---

7. CAN Bus-Off

Checked directly against the CAN controller's error status:

```
if (HAL_CAN_GetError(&hcan1) & HAL_CAN_ERROR_BOF)
    can_busoff_fault = 1;
```

Device	Action
Fan	OFF
Buzzer	Continuous
LCD	CAN BUS-OFF

---

8. Sensor Failure

Flags impossible readings coming in over CAN, this catches cases where STM1's own validation missed something or a value got corrupted in transit:

```
if (temp < -20 || temp > 100)
    temp_sensor_fault = 1;

if (voltage < 0 || voltage > 20)
    voltage_sensor_fault = 1;

if (current < -15 || current > 15)
    current_sensor_fault = 1;
```

Device	Action
Fan	OFF
Buzzer	Fast beep

LCD	SENSOR FAILURE

Fault Priority

If several faults are active at once, only the highest-priority one is shown on the LCD:

Priority	Fault	Fan	Buzzer	LCD
1	Smoke detected	ON	Continuous	FIRE RISK
2	CAN bus-off	OFF	Continuous	CAN BUS-OFF
3	CAN timeout	OFF	Continuous	CAN TIMEOUT
4	Critical temperature (>55°C)	ON	Slow beep	BATTERY HOT
5	Over voltage	OFF	Continuous	OVER VOLTAGE
6	Over current	OFF	Slow beep	OVER CURRENT
7	Under voltage	OFF	Beep every 2 sec	UNDER VOLTAGE
8	Over temperature (45-55°C)	ON	OFF	COOLING...
9	Sensor failure	OFF	Fast beep	SENSOR FAILURE
10	No fault	OFF	OFF	Live sensor data

Complete STM2 Program Flow

```
Receive CAN Data
      |
Calculate Data
(Power, SOC, Load, Battery Status)
      |
Check Under Voltage
      |
Check Over Voltage
      |
Check Over Current
      |
Check Over Temperature
      |
Check Smoke
      |
Check CAN Timeout
      |
Check CAN Bus-Off
      |
Check Sensor Failure
      |
Select Highest Priority Fault
      |
Control Fan
      |
Control Buzzer
      |
Update LCD
```

Each fault is checked independently, then a priority manager decides which single warning gets shown and which actions (fan, buzzer, LCD) get taken. This mirrors how fault handling is typically structured in automotive ECUs, keeping detection and response logic clean and modular.